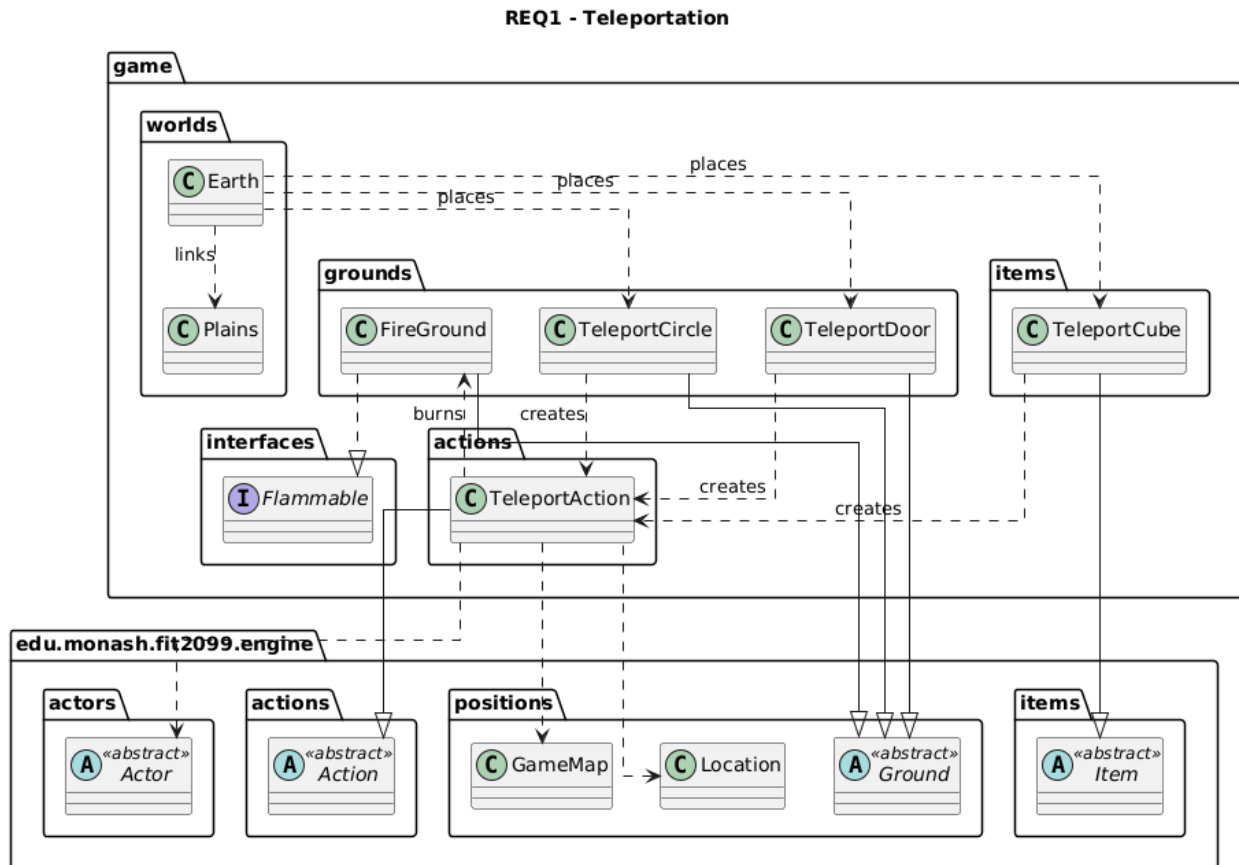


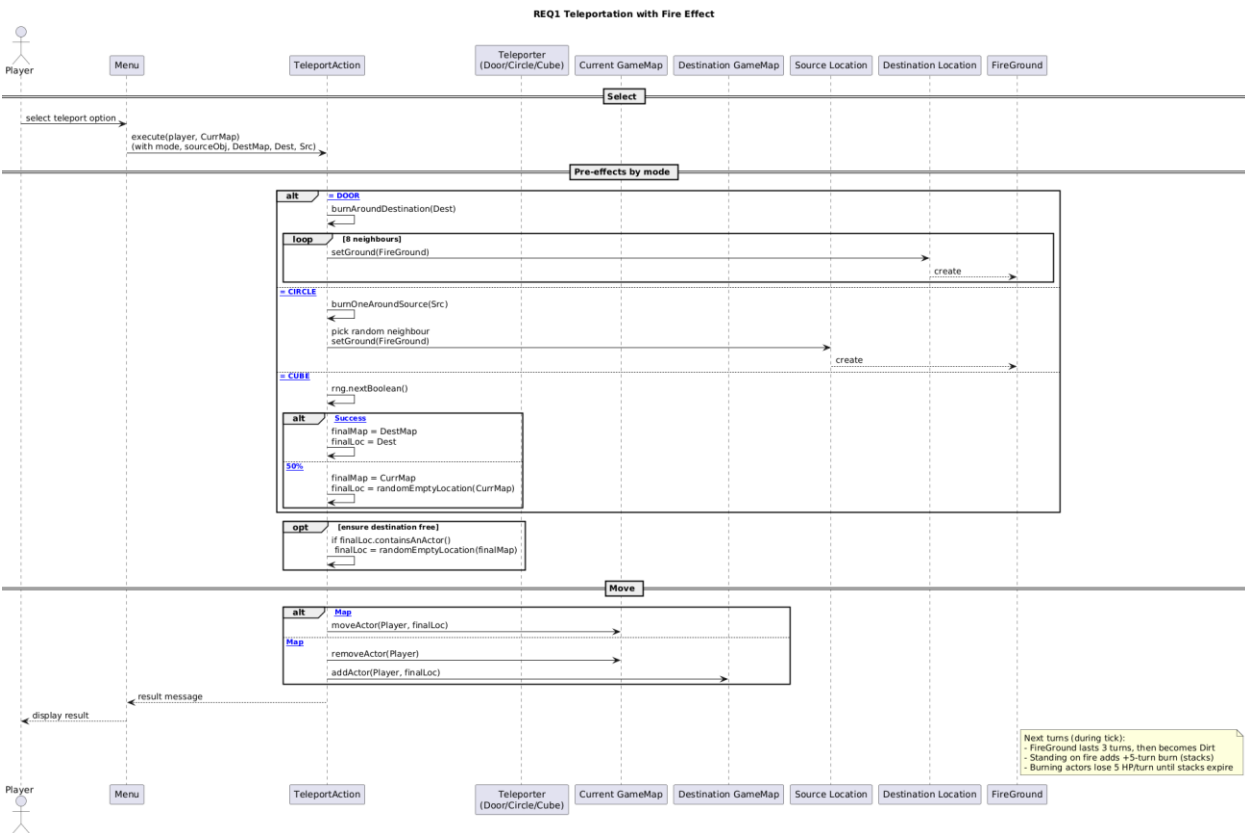
FIT2099 Assignment 2 – REQ1

Alex Bui - 34662901

UML Diagram



Sequence Diagram



Design Rationale

The design introduces a modular teleportation system that enables the Explorer to travel between and within maps using three given teleportation mechanisms: TeleportDoor, TeleportCircle, and TeleportCube. Each teleportation type is represented as an independent class, implementing unique behaviors while sharing a common execution logic through the TeleportAction class.

The TeleportAction centralises teleportation mechanics, handling actor transfer, malfunction probability (for the cube), and environmental effects such as fire ignition. This decouples teleport logic from specific ground or item classes, promoting high cohesion and low coupling. The FireGround class models temporary fire tiles that apply stackable burn effects, tracked globally via a WeakHashMap for efficiency.

The world is composed of multiple maps (Earth and Plains), with teleport destinations linked dynamically during world construction. Each TeleportDoor and TeleportCircle maintains a list of destination locations, allowing two-way or multi-map teleportation without hardcoding dependencies. The TeleportCube, as a portable item, extends the teleport system beyond static grounds.

Advantages of the Design:

- **Single Responsibility Principle (SRP)**
 - Each class performs a distinct function:
 - TeleportDoor, TeleportCircle, and TeleportCube handle destination registration and teleport-specific effects.
 - TeleportAction executes the teleportation itself and manages fire or malfunction logic.
 - FireGround is responsible solely for burn persistence and damage application.
- **Open-Closed Principle (OCP)**
 - The design is open for extension but closed for modification. New teleportation mechanisms (i.e. with different associated side-effects, functionalities) can be introduced by extending TeleportAction.Mode and/or creating new ground/item classes without altering the existing codebase.
- **Dependency Inversion Principle (DIP)**

- TeleportAction depends on abstract interfaces (e.g., Ground, Item) rather than concrete implementations. This abstraction allows both static and portable teleporters to share the same action logic, reducing coupling between high-level and low-level modules.
- **Low Coupling and High Cohesion**
 - Each teleport-related class encapsulates its own data and behavior. For example, *TeleportCube* handles endpoints independently of the maps, and *TeleportAction* operates uniformly across all teleport types. This independence supports parallel development and easy integration into new worlds.
- **Extensibility and Reusability**
 - The modular TeleportAction architecture allows reuse of teleport logic across various triggers (e.g., future portals, spells). Similarly, FireGround can be reused for other environmental hazards (lava, acid, etc.), improving flexibility and maintainability.
- **Connascence**
 - The system primarily exhibits Connascence of Type and Connascence of Meaning, where related classes (e.g., TeleportDoor, TeleportCircle, TeleportCube) share consistent parameter types and semantic contracts with TeleportAction. This ensures that if method signatures or teleport modes evolve, dependent modules remain aligned. Stronger forms such as Connascence of Execution Order are intentionally avoided, with actions are processed generically and independently, reducing hidden dependencies and improving modifiability.

Disadvantages of the Design:

- **Potential Duplication for New Teleports:** Future teleport types that require unique effects (e.g., frost, lightning) may reintroduce minor duplication in action logic, partially reducing OCP compliance.
- **Potential Duplication for New Teleports:** Future teleport types that require unique effects (e.g., frost, lightning) may reintroduce minor duplication in action logic, partially reducing OCP compliance.
- **Inter-Map Link Maintenance:** The current linking destination coordinates between multiple maps may introduce additional complexity, risk of inconsistency during expansion. This approach also creates more potential breakpoints during debugging, particularly as world size and teleport count increase.

Conclusion

The teleportation system demonstrates a strong balance between SOLID principles, DRY, and connascence awareness, ensuring modularity, scalability, and maintainability. The decoupled teleport behavior is structured through cohesive yet loosely connected classes, allowing the design to support future expansion with minimal refactoring. This results in a clean, extensible, and immersive teleportation mechanism that aligns with the FIT2099 engine's object-oriented architecture and meets requirements' standards for design clarity, reasoning, and principles compliance.